FRAPPE ERPNEXT USB DIGITAL SIGNATURE PROJECT

A complete Technical Blueprint for Multi-Sign PDF Implementation using USB Tokens

1. Project Overview & Architecture

Yeh project aapko browser (Frappe Desk) ke andar directly Hardware USB Digital Signature Token (jaise ePass2003, ProxKey, mToken) ke zariye safe, Adobe-compliant, aur legally valid multiple digital signatures lagane ki functional capability deta hai. Kyuki web browser security limitations ki wajah se direct hardware/ USB access nahi kar sakta, hum ek hybrid approach use karte hain.

System Workflow Architecture

Is system me total teen major layers kaam karti hain:

- **Frappe Backend Server (Linux/Cloud):** Yeh PDF file generate karta hai, `pyHanko` library se byte-range placeholders banata hai, document hash calculate karta hai, aur end me response cryptographic token se aane ke baad signature inject karta hai.
- **Frappe UI Client (Browser JS):** Yeh User Interface manage karta hai, button click handle karta hai, server se hash laakar local machine service ko bhejta hai, aur local machine se aane wale signed bytes server par push karta hai.
- **Local Signing Helper (Windows Service):** Yeh user ke local system par background me `127.0.0.1:39999` par chalne wali FastAPI utility hai jo native PKCS#11 drivers se handle karti hai aur USB token ke cryptographic core se communication karwati hai.

2. Technical Prerequisites & Frameworks

Project start karne se pehle aapko in technologies aur concepts ko setup karna hoga:

Component/Library	Purpose	Key Skill / Area to Study
Frappe Framework / ERPNext	Server & UI Platform	Custom App lifecycle, Whitelisted API structure, Client-side scripts.
pyHanko (Python)	PDF Signatures Core	Incremental PDF modification, PAdES standard format, Byte-range placement.
python-pkcs11 (Python)	Hardware Interaction	Cryptographic PKCS#11 API slots, Token logins, raw signing mechanisms.
FastAPI (Python Windows)	Local REST Server	CORS policy execution, handling local requests, PyInstaller production build.

3. Component 1: Frappe Custom App Backend (`dsc_signing`)

Aapko Frappe workspace me ek naya app create karna hoga (`bench new-app dsc_signing`). Is app me hum backend hooks aur specific custom API targets configure karenge.

DocType Schema Configuration

Aapko custom application ke andar ye teen important DocTypes setup karne padenge:

1. **DSC Signature Request:** Yeh workflow pipeline state ko trace karega (Fields: `reference_doctype`, `reference_name`, `current_stage`, `original_file`, `signed_file`).
2. **DSC Signer Profile:** Har authenticated corporate user ka token structure save rakhega (Fields: `user`, `certificate_serial`, `subject_dn`).
3. **DSC Audit Log:** Yeh ek immutable read-only logger hoga jo system activity security audits track karega.

Server Logic Implementation (`api.py`)

```
import frappe
from io import BytesIO
from pyhanko.pdf_utils.incremental_writer import IncrementalWriter
from pyhanko.sign import fields, signers

@frappe.whitelist()
def prepare_for_signing(doctype, docname, signature_round):
    # 1. Fetch appropriate PDF source content based on signing round
    if str(signature_round) == "1":
        pdf_bytes = frappe.get_print(doctype, docname, as_pdf=True)
    else:
        # Load output of previous signature round
        req_doc = frappe.get_doc("DSC Signature Request", {"reference_name": docname})
        pdf_bytes = req_doc.get_pdf_content()

    # 2. Setup an IncrementalWriter instance to preserve structural layout
    w = IncrementalWriter(BytesIO(pdf_bytes))
    field_name = f"Signature_Field_{signature_round}"

    # Position logic: Round 1 (Left), Round 2 (Right)
    box_coord = (50, 50, 200, 150) if str(signature_round) == "1" else (350, 50, 500, 150)

    # Inject signature layout structure
    fields.append_signature_field(
        w, sig_field_spec=fields.SigFieldSpec(sig_field_name=field_name, box=box_coord)
    )

    meta = signers.PdfSignatureMetadata(field_name=field_name)
    pdf_signer = signers.PdfSigner(meta)

    # Ready current stream execution environment context
    prep_res = pdf_signer.prepare_hash_digest(w)

    # Store dynamic background task metadata state safely
    save_interim_task_state(docname, signature_round, prep_res)
```

```
    return {
        "hash_to_sign": prep_res.md_ready.hex(),
        "signature_round": signature_round
    }

@frappe.whitelist()
def embed_signature(docname, signature_bytes, signature_round):
    # Retrieve prepared operational context streams
    # Use pyHanko next() logic pipelines to embed signature bytes directly inside layout
    # Apply LTV (Long Term Validation) with active TSA server mapping counters
    return {"status": "Success"}
```

4. Component 2: Browser Glue UI Layer (Client Script)

Yeh custom JavaScript block aapko ERPNext workspace interface ke upar action control interface mappings inject karne me madad karta hai.

```
frappe.ui.form.on('Sales Invoice', {
    refresh: function(frm) {
        if (frm.doc.docstatus === 1) { // Submitted invoices only
            if (!frm.doc.custom_signature_1_status) {
                frm.add_custom_button(__('Sign as Maker (DSC)'), function() {
                    execute_dsc_workflow(frm, "1");
                }, __('Digital Signature'));
            } else if (frm.doc.custom_signature_1_status && !
frm.doc.custom_signature_2_status) {
                frm.add_custom_button(__('Sign as Checker (DSC)'), function() {
                    execute_dsc_workflow(frm, "2");
                }, __('Digital Signature'));
            }
        }
    }
});

async function execute_dsc_workflow(frm, round) {
    frappe.show_alert({message: __('Preparing document context...'), indicator: 'blue'});

    let prep = await frappe.xcall('dsc_signing.api.prepare_for_signing', {
        doctype: frm.doc.doctype,
        docname: frm.doc.name,
        signature_round: round
    });

    try {
        let local_res = await fetch('https://127.0.0.1:39999/sign', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ hash: prep.hash_to_sign })
        });
        let sign_bytes = await local_res.json();
```

```
        await frappe.xcall('dsc_signing.api.embed_signature', {
            docname: frm.doc.name,
            signature_bytes: sign_bytes.signature,
            signature_round: round
        });

        frappe.msgprint(__('Signature applied successfully!'));
        frm.reload_doc();
    } catch(err) {
        frappe.msgprint(__('Error: Windows Local Signer service is disconnected. Please
launch the helper utility.'));
    }
}
```

5. Component 3: Local Windows Service Helper (FastAPI Client)

Local helper Windows PC runtime workspace engine environment host machine par operate karega jo middleware bridge pipelines facilitate karta hai.

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
import pkcs11

app = FastAPI()

# Strict security isolation parameters
app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://your-company-erpnext.com"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.post("/sign")
def sign_hash_payload(payload: dict):
    target_hash = bytes.fromhex(payload['hash'])

    # Auto detection matrix mapping path references
    pkcs11_driver_path = r"C:\Windows\System32\eps2003csp11.dll" # ePass2003 Target Path
Reference

    lib = pkcs11.lib(pkcs11_driver_path)
    slots = lib.get_slots()

    if not slots:
        return {"error": "No USB Token Hardware detected."}

    token = slots[0].get_token()
    with token.open(user_pin="12345678") as session:
        priv_key = session.get_key(pkcs11.ObjectClass.PRIVATE_KEY)
        signature = priv_key.sign(target_hash)
```

```
    return {"signature": signature.hex()}
```

6. Multi-Signature Structural Ground Rules (Adobe Compliance)

> **CRITICAL NOTICE FOR PRODUCTION IMPLEMENTATION:**
>
> Jab multi-sign digital layouts code design implement kar rahe hon, toh niche likhe cryptographic constraints follow karna mandatory hai, warna Adobe Acrobat signature corrupted ya invalid show karega:
>
> - **Mandatory Incremental Updates:** Python stack file generation pipeline workflows me hamesha `IncrementalWriter` use karein. Purane layout object blocks par structural overwrite block paths crash prevent karne ke liye system files update state append mode me render honi chahiye.
> - **Unique Field Identity Mappings:** Signature structural parameters attributes mapping blocks dynamic references distinct hone zaroori hain (e.g., `Signature_Field_1` and `Signature_Field_2`).
> - **Zero Overlay Coordinate Layout Geometry:** Har layer signature field visual frame block box configuration separation spacing structure alag hone chahiye taaki validation state properties collision abstract levels par completely prevent ho sake.

7. Phase-wise Implementation Strategy (Roadmap)

Aapko zero se start karke complete system deployment tak is phase pipeline process structure sequence ko rigorously execution pattern me follow karna chahiye:

1. **Phase 1 (Isolated Cryptographic Testing):** Local workspace terminal environment script parameters control system layout standard pipelines verify karein. Pehle ERPNext engine framework layers integrate kiye bina offline PDF stream binary signatures verify karein.
2. **Phase 2 (FastAPI Local Deployment Testing):** Postman payload request parameters test engine execute karein jo cross execution environment interfaces verify karega.
3. **Phase 3 (Frappe Custom Structural Modules Integration):** Frappe custom app layers controller models implement karein.
4. **Phase 4 (End-to-End Core Verification):** Frontend client scripts trigger runtime state pipeline integrations fully execution loop trace check run karein.
5. **Phase 5 (Production Binaries Compilation):** Local client utilities binary modules `PyInstaller --onefile` production builds compile karke user system environment installers deploy karein.